

LAB | WEEKS 1 + 2

Start CampusEats: Git, Components, Props and State

One session, two weeks. Set up your repository, build the Week 1 components, then make them data-driven with props and state.

You will end with	CampusEats on GitHub showing YOUR restaurant's menu, switchable vendors and a working cart badge, tagged week-1 and week-2
Time	About 2.5 hours
You need	Node.js, Git and VS Code installed; a GitHub account you can sign in to; campuseats-starter.css from iTa'LeeM

Checklist

Tick each box as you go. Show the lecturer or demonstrator the last box before you leave.

- ☐ A1 Tools print version numbers
- ☐ A2 Git knows your name and email
- ☐ A3 Empty repository created on GitHub
- ☐ A4 Vite + React app runs at localhost:5173
- ☐ A5 Starter cleaned, CampusEats CSS added
- ☐ A6 First push: files visible on GitHub
- ☐ A7 Lecturer (qawyhamz) added as collaborator
- ☐ A8 AI_USAGE.md created in the project root
- ☐ B1 Header built (type-along)
- ☐ B2 VendorCard built (on your own)
- ☐ B3 MenuItemCard and Footer added, App shows all four
- ☐ B4 Week 1 AI log + REFLECTION.md written, tag week-1 pushed
- ☐ C1 Your restaurant's menu shows from data/vendors.js
- ☐ C2 Clicking a vendor switches the menu
- ☐ C3 Add to cart updates the badge
- ☐ C4 Week 2 AI log written, tag week-2 pushed and shown to the lecturer

Part A: Git and the project (about 60 min)

A1 Check your tools

Open a terminal (VS Code: Terminal → New Terminal):

```
node -v
npm -v
git --version
```

Each prints a version number. "Not recognised"? Close and reopen the terminal, then ask for help.

A2 Tell Git who you are

Use the same email as your GitHub account:

```
git config --global user.name "Your Full Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

A3 Create an empty repository on GitHub

1. Sign in at **github.com** → + (top right) → **New repository**.
2. Name: **campuseats-~~<matric no>~~**, e.g. **campuseats-2215678**.
3. Public or Private, either is fine (you will add the lecturer in A7).
4. **Leave README, .gitignore and licence unticked**. The repository must be empty.
5. Click **Create repository** and keep the page open; you need its URL in A6.

A4 Create the React project

Go to the folder where you keep coursework, then:

```
cd Documents
npm create vite@latest campuseats -- --template react
```

Question	Answer
Ok to proceed? (y)	y
Which linter to use?	ESLint
Install with npm and start now?	No

```
cd campuseats
npm install
npm run dev
```

✓ **Checkpoint:** <http://localhost:5173> shows the Vite + React page. Keep this terminal running; open a second terminal for Git.

A5 Clean the starter and add the CSS

1. Open the folder in VS Code: **code .**
2. Delete **src/App.css** and the **src/assets** folder.
3. Open **src/index.css**, delete everything, paste in **campuseats-starter.css**.
4. Replace everything in **src/App.jsx** with the code below.
5. Create an empty folder **src/components**.

src/App.jsx

```
function App() {
  return <h1>CampusEats</h1>
}

export default App
```

A6 First commit and push

In the second terminal, inside the **campuseats** folder. Use the URL from A3:

```
git init
git add .
git commit -m "Initial Vite + React project"
git branch -M main
git remote add origin https://github.com/<your-username>/campuseats-<matric no>.git
git push -u origin main
```

First push: a browser window asks you to sign in to GitHub. Choose **Sign in with your browser** and approve.

✓ **Checkpoint:** Refresh your repository page on GitHub: your files are there.

A7 Add the lecturer as a collaborator

So the lecturer can see and mark your repository (required, public or private):

1. On your repository page: **Settings** → **Collaborators** (left menu).
2. Click **Add people**, type **qawyhamz**, select **qawyhamz**, then **Add qawyhamz to this repository**.
3. It shows as "Pending invite" until the lecturer accepts. That is fine; carry on.

A8 Start your AI log

You may use AI tools (ChatGPT, Copilot, Claude, Gemini...). What is assessed is that you understand your own code, so you keep a log every week. **Using AI never costs marks; a vague or missing log does.**

Download **AI_USAGE.md** from iTa'LeeM and put it in the project root, next to **package.json** (or create the file and paste this):

AI_USAGE.md (project root)

```
# AI usage log

## Week 1
- Tool(s):
- What I asked for:
- What I kept, changed or rejected, and why:
- One thing the AI got wrong and how I fixed it:

## Week 2
- Tool(s):
- What I asked for:
- What I kept, changed or rejected, and why:
- One thing the AI got wrong and how I fixed it:
```

Did not use AI this week? Write "None" under Tool(s). Then commit:

```
git add .
git commit -m "Add AI usage log"
git push
```

Part B: Week 1 components (about 25 min)

A **component** is a function that returns JSX (HTML-like code). Each one lives in its own file in **src/components** and is exported so App can use it.

B1 Header (type along with the lecturer)

src/components/Header.jsx

```
function Header() {
  const cartCount = 0 // becomes state in Part C

  return (
    <header className="header">
      <h1 className="logo">CampusEats</h1>
      <nav className="nav">
        <a href="#">Vendors</a>
        <a href="#">My Orders</a>
        <a href="#" className="cart">
          Cart <span className="badge">{cartCount}</span>
        </a>
      </nav>
    </header>
  )
}

export default Header
```

Notice: `className` not `class`; `{cartCount}` puts a JavaScript value into the page.

B2 VendorCard (on your own)

Type this yourself. Change the vendor details to **your own restaurant**.

src/components/VendorCard.jsx

```
function VendorCard() {
  const vendor = {
    name: 'Kafe Mahallah Ali',
    location: 'Mahallah Ali, Block C',
    openHours: '7:00 am - 10:00 pm',
    isOpen: true,
  }

  return (
    <article className="card vendor-card">
      <div className="thumb" aria-hidden="true">
        {vendor.name.charAt(0)}
      </div>
      <div>
        <h2>{vendor.name}</h2>
        <p className="muted">{vendor.location}</p>
        <p className="muted">Open: {vendor.openHours}</p>
        <span className={vendor.isOpen ? 'status open' : 'status closed'}>
          {vendor.isOpen ? 'Open now' : 'Closed'}
        </span>
      </div>
    </article>
  )
}

export default VendorCard
```

`condition ? A : B` shows A when the condition is true, otherwise B.

B3 MenuItemCard and Footer (copy, then read)

src/components/MenuItemCard.jsx

```
function MenuItemCard() {
  const item = {
    name: 'Nasi Lemak Ayam',
    description: 'Coconut rice, fried chicken, sambal, egg and peanuts',
    price: 7.5,
    available: true,
  }

  return (
    <article className="card menu-card">
      <div className="thumb" aria-hidden="true">{item.name.charAt(0)}</div>
      <h3>{item.name}</h3>
      <p className="muted">{item.description}</p>
      <p className="price">RM {item.price.toFixed(2)}</p>
      <button className="btn" disabled={!item.available}>
        {item.available ? 'Add to cart' : 'Sold out'}
      </button>
    </article>
  )
}

export default MenuItemCard
```

src/components/Footer.jsx

```
function Footer() {
  return (
    <footer className="footer">
      <p>&copy; {new Date().getFullYear()} CampusEats &middot; BICS 3301, IIUM</p>
    </footer>
  )
}

export default Footer
```

Now put all four on the page:

src/App.jsx

```
import Header from './components/Header.jsx'
import VendorCard from './components/VendorCard.jsx'
import MenuItemCard from './components/MenuItemCard.jsx'
import Footer from './components/Footer.jsx'

function App() {
  return (
    <>
      <Header />
      <main className="container">
        <section>
          <h2 className="section-title">Today's vendors</h2>
          <VendorCard />
        </section>
        <section>
          <h2 className="section-title">Popular items</h2>
          <div className="grid">
            <MenuItemCard />
          </div>
        </section>
      </main>
      <Footer />
    </>
  )
}

export default App
```

`<>...</>` (a fragment) wraps everything, because a component must return **one** element.

✓ **Checkpoint:** The page shows the header, one vendor card, one menu card and the footer.

B4 Log, reflect, then tag Week 1

1. Fill in the **Week 1** section of `AI_USAGE.md`.
2. Create **REFLECTION.md** in the project root (template on iTa'LeeM) and answer briefly, 1–3 sentences each:

REFLECTION.md (project root)

```
# Week 1 reflection

1. What is the difference between building a UI imperatively (plain DOM code) and declaratively (React)?

2. Why must a component name start with a capital letter?

3. What does a fragment <>...</> do, and why not just use a <div>?

4. Name one benefit of splitting the UI into small components.
```

Then commit, push and tag:

```
git add .
git commit -m "Week 1: components, AI log and reflection"
git push
git tag week-1
git push origin week-1
```

Part C: Week 2, props and state (about 55 min)

Word	Meaning
Prop	What a component is given by its parent. Read-only.
State	What a component remembers . Changing it with the setter re-renders the screen.
Event	What the user does (a click). Handled with <code>onClick={...}</code> .

C1 Your menu from data

Create `src/data/vendors.js`. Make the **first vendor your own restaurant** with at least 3 items, and keep one more vendor:

`src/data/vendors.js`

```
const vendors = [
  {
    id: 'my-restaurant',
    name: 'Your Restaurant Name',
    location: 'Mahallah ..., Block ...',
    openHours: '7:00 am - 10:00 pm',
    isOpen: true,
    menu: [
      { id: 'my-1', name: 'Dish 1', description: '...', price: 7.5, category: 'Rice',
        available: true },
      { id: 'my-2', name: 'Dish 2', description: '...', price: 6, category:
        'Noodles', available: true },
      { id: 'my-3', name: 'Drink 1', description: '...', price: 2.5, category:
        'Drinks', available: false },
    ],
  },
  {
    id: 'kafe-aminah',
    name: 'Kafe Mahallah Aminah',
    location: 'Mahallah Aminah, Ground Floor',
    openHours: '8:00 am - 9:00 pm',
    isOpen: true,
    menu: [
      { id: 'ami-1', name: 'Nasi Ayam Penyet', description: 'Smashed fried chicken
        with sambal and rice', price: 9, category: 'Rice', available: true },
      { id: 'ami-2', name: 'Air Bandung', description: 'Rose syrup with milk', price:
        3, category: 'Drinks', available: true },
    ],
  },
]

export default vendors
```

Change **MenuItemCard** to receive its item as a **prop** (delete the hard-coded **item** object):

src/components/MenuItemCard.jsx

```
function MenuItemCard({ item }) {
  return (
    <article className="card menu-card">
      <div className="thumb" aria-hidden="true">{item.name.charAt(0)}</div>
      <p className="tag">{item.category}</p>
      <h3>{item.name}</h3>
      <p className="muted">{item.description}</p>
      <p className="price">RM {item.price.toFixed(2)}</p>
      <button className="btn" disabled={!item.available}>
        {item.available ? 'Add to cart' : 'Sold out'}
      </button>
    </article>
  )
}

export default MenuItemCard
```

Create **MenuList**, which turns the array into cards with **.map()**:

src/components/MenuList.jsx

```
import MenuItemCard from './MenuItemCard.jsx'

function MenuList({ items }) {
  if (items.length === 0) {
    return <p className="muted">No items on this menu yet.</p>
  }

  return (
    <div className="grid">
      {items.map((item) => (
        <MenuItemCard key={item.id} item={item} />
      ))}
    </div>
  )
}

export default MenuList
```


In **App.jsx**, show the first vendor's menu:

src/App.jsx

```
import vendors from './data/vendors.js'
import Header from './components/Header.jsx'
import MenuList from './components/MenuList.jsx'
import Footer from './components/Footer.jsx'

function App() {
  const selectedVendor = vendors[0]

  return (
    <>
      <Header />
      <main className="container">
        <section>
          <h2 className="section-title">Menu: {selectedVendor.name}</h2>
          <MenuList items={selectedVendor.menu} />
        </section>
      </main>
      <Footer />
    </>
  )
}

export default App
```

✓ **Checkpoint:** Your restaurant's dishes appear as cards. The unavailable one says Sold out.
Commit: `git add . → git commit -m "Menu from data with map" → git push.`

C2 Click a vendor to switch the menu

First, paste these Week 2 styles at the **end** of **src/index.css** (vendor grid, clickable cards, highlight, category tag):

src/index.css (add at the end)

```
/* Week 2 additions */
.vendor-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 1rem; }
button.card { font: inherit; color: inherit; text-align: left; border: 2px solid transparent; cursor: pointer; width: 100%; }
button.card:hover { border-color: var(--green-light); }
.vendor-card.selected { border-color: var(--green); }
.card h3 { font-size: 1.05rem; }
.tag { display: inline-block; margin: 0 0 0.3rem; padding: 0.05rem 0.5rem; border-radius: 999px; background: var(--green-light); color: var(--green); font-size: 0.75rem; font-weight: 600; }
```

Change **VendorCard** to receive its vendor and a click function as **props**:

`src/components/VendorCard.jsx`

```
function VendorCard({ vendor, isSelected, onSelect }) {
  return (
    <button
      type="button"
      className={isSelected ? 'card vendor-card selected' : 'card vendor-card'}
      onClick={() => onSelect(vendor.id)}
    >
      <div className="thumb" aria-hidden="true">{vendor.name.charAt(0)}</div>
      <div>
        <h3>{vendor.name}</h3>
        <p className="muted">{vendor.location}</p>
        <p className="muted">Open: {vendor.openHours}</p>
        <span className={vendor.isOpen ? 'status open' : 'status closed'}>
          {vendor.isOpen ? 'Open now' : 'Closed'}
        </span>
      </div>
    </button>
  )
}

export default VendorCard
```

In **App.jsx**, remember which vendor is selected with **state** and show one card per vendor:

src/App.jsx

```
import { useState } from 'react'
import vendors from './data/vendors.js'
import Header from './components/Header.jsx'
import VendorCard from './components/VendorCard.jsx'
import MenuList from './components/MenuList.jsx'
import Footer from './components/Footer.jsx'

function App() {
  const [selectedVendorId, setSelectedVendorId] = useState(vendors[0].id)
  const selectedVendor = vendors.find((v) => v.id === selectedVendorId)

  return (
    <>
      <Header />
      <main className="container">
        <section>
          <h2 className="section-title">Choose a vendor</h2>
          <div className="vendor-grid">
            {vendors.map((vendor) => (
              <VendorCard
                key={vendor.id}
                vendor={vendor}
                isSelected={vendor.id === selectedVendorId}
                onSelect={setSelectedVendorId}
              />
            ))}
          </div>
        </section>
        <section>
          <h2 className="section-title">Menu: {selectedVendor.name}</h2>
          <MenuList items={selectedVendor.menu} />
        </section>
      </main>
      <Footer />
    </>
  )
}

export default App
```

 **Checkpoint:** Clicking the other vendor changes the menu and highlights that card. Commit and push: "Switch vendors with state".

C3 Add to cart

The cart lives in **App** (the parent). Children get a function **onAdd** as a prop and call it on click.

1. MenuItemCard: receive **onAdd** and call it on click (two lines change):

src/components/MenuItemCard.jsx (changes only)

```
function MenuItemCard({ item, onAdd }) {
  ...
  <button className="btn" disabled={!item.available} onClick={() => onAdd(item)}>
```

2. MenuList: pass `onAdd` on to every card:`src/components/MenuList.jsx (changes only)`

```
function MenuList({ items, onAdd }) {
  ...
  <MenuItemCard key={item.id} item={item} onAdd={onAdd} />
```

3. Header: receive `cartCount` as a prop. Delete the line `const cartCount = 0`:`src/components/Header.jsx (changes only)`

```
function Header({ cartCount }) {
```

4. App: add cart state and the function, then pass them down:`src/App.jsx (additions)`

```
const [cart, setCart] = useState([])

function handleAddToCart(item) {
  setCart((prevCart) => [...prevCart, item]) // a NEW array, never cart.push()
}

// in the JSX:
<Header cartCount={cart.length} />
...
<MenuList items={selectedVendor.menu} onAdd={handleAddToCart} />
```

✓ **Checkpoint:** Clicking Add to cart increases the badge in the header. Sold-out items cannot be clicked. Commit and push: "Cart with onAdd".

C4 Log, tag Week 2 and sign offFill in the **Week 2** section of `AI_USAGE.md`, then:

```
git add .
git commit -m "Week 2: vendor data, props, lists and cart count"
git push
git tag week-2
git push origin week-2
```

Show the lecturer or demonstrator: your app running, both tags (week-1, week-2) under **Tags** on GitHub, `AI_USAGE.md` and `REFLECTION.md` in the repo, and qawyhamz listed under **Settings → Collaborators**.

Not finished? Finish at home and push the week-2 tag by **Sunday 23:59**. Catch-up code for each checkpoint is on iTa'LeeM after class.

If something goes wrong

You see	Fix
<code>npm / git</code> not recognised	Close and reopen the terminal
Tagged before adding the log or reflection	Commit and push the files, then move the tag: <code>git tag -d week-1, git push origin :refs/tags/week-1, git tag week-1, git push origin week-1</code>
<code>fatal: not a git repository</code>	You are in the wrong folder: <code>cd campuseats</code>
<code>Author identity unknown</code>	Do step A2, then commit again

You see	Fix
Push rejected (fetch first)	Repo was created with a README: <code>git pull origin main --allow-unrelated-histories</code> , then <code>git push -u origin main</code>
Permission denied / 403	PC signed in to another GitHub account: Windows Credential Manager → Windows Credentials → remove github.com, push again
remote origin already exists	<code>git remote set-url origin <your-url></code>
Blank white page	Open the browser console (F12) and read the red error; usually a typo or missing import
Failed to resolve import	File name or path in the import does not match the file (capital letters matter)
"Each child in a list should have a unique key"	Add <code>key={item.id}</code> (or <code>vendor.id</code>) inside <code>.map()</code>
Button runs immediately / badge jumps on load	You wrote <code>onClick={onAdd(item)}</code> ; it must be <code>onClick={() => onAdd(item)}</code>
Badge does not change	Check you used <code>setCart</code> , not <code>cart.push</code> , and passed <code>cartCount={cart.length}</code> to Header

Before you leave a lab PC

- ☐ Everything pushed (`git status` says "nothing to commit")
- ☐ Signed out of GitHub in the browser
- ☐ Credential Manager: github.com entry removed (so the next class cannot push to your account)